

Deployment and Setup Guide

Chat With Docs Image

A practical guide for local setup, offline mode, Docker, Render deployment, authentication configuration, data persistence, and production checks.

Prepared 03 October 2026

Quick conclusion

The project runs as a Python 3.11 Streamlit application. Local development can use Gemini Cloud or Ollama Offline. Docker includes Poppler and Tesseract. Render can run the container with a persistent /app/data disk, but production deployments should also configure OIDC, external backups, resource monitoring, and a secret manager.

Deployment options

Option	Best for	Data location
Local Streamlit	Development, demos, private use	Project directory or APP_DATA_DIR
Docker	Repeatable local or server deployment	/app/data volume
Render	Small hosted MVP	Persistent Render disk at /app/data

1. Prerequisites

Install the following before starting:

- Python 3.11.
- Git, if cloning the repository.
- Tesseract OCR for image and scanned-PDF extraction when running outside Docker.
- Poppler utilities for PDF page rendering when running outside Docker.
- A Google Gemini API key for cloud AI, or Ollama plus local models for offline AI.

The Dockerfile installs Tesseract and Poppler automatically. On Windows, the application checks the standard Tesseract installation path at C:\Program Files\Tesseract-OCR\tesseract.exe.

2. Local setup

1. Create or open the project directory.
2. Create a Python virtual environment.
3. Install the requirements.
4. Copy .env.example to .env and set the provider configuration.
5. Start Streamlit.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
streamlit run app.py
```

On macOS or Linux, activate with source venv/bin/activate. The first Gemini indexing or chat request requires GOOGLE_API_KEY. For local-only use, choose Ollama Offline in the AI Provider and Ranking panel.

3. Environment variables and limits

Variable	Default	Purpose
GOOGLE_API_KEY	None	Gemini chat and embedding access
APP_DATA_DIR	.	Root for collections, backups, auth, and workspaces
MAX_UPLOAD_SIZE_MB	50	Maximum uploaded file and ZIP size
MAX_IMPORT_ZIP_MEMBERS	100	Maximum files listed in an imported collection ZIP
MAX_IMPORT_ZIP_UNCOMPRESSED_MB	250	Maximum expanded ZIP size
MAX_PDF_PAGES	200	Maximum PDF page count per extraction
PROCESSING_TIMEOUT_SECONDS	180	Cooperative document processing limit
OLLAMA_HOST	Optional	Ollama service URL when configured externally

The Streamlit upload cap is set to 50 MB in .streamlit/config.toml. The application-level limits can be lowered through .env for smaller deployments.

4. Ollama Offline setup

Install Ollama separately, start the service, and pull both a chat model and an embedding model. The default application values are llama3.2 and nomic-embed-text.

```
ollama pull llama3.2
ollama pull nomic-embed-text
```

Then open the application, select Ollama Offline, confirm the two model names, and process files. Indexing and chat will use the local service instead of Gemini.

5. Application workflow

Step	What happens
Upload	The user selects supported files and optionally selects a subset to process.
Extract	The app parses text, tables, slides, spreadsheets, or OCR content.
Chunk	Content is split into chunks and annotated with filename, type, page, and chunk metadata.
Index	Embeddings are stored in a native FAISS index and chunks are persisted as JSON.
Retrieve	Vector and BM25 keyword results are merged and optionally reranked.
Generate	Gemini or Ollama receives selected context and produces a cited answer or summary.
Manage	Users can preview, tag, categorize, rename, delete, reindex, export, and compare documents.

6. Docker setup

Build and run the container with a persistent data directory. The container exposes port 8501 and sets APP_DATA_DIR=/app/data.

```
docker build -t chat-with-docs-image .
docker run --rm -p 8501:8501 \
  -e GOOGLE_API_KEY=your_key \
  -v chat-with-docs-data:/app/data \
  chat-with-docs-image
```

Open <http://localhost:8501>. The named volume preserves SQLite authentication, collections, FAISS indexes, chat history, and backups across container restarts.

7. Render deployment

1. Push the repository to a Git provider connected to Render.
2. Create a Render Web Service using the included render.yaml or choose Docker as the runtime.
3. Set GOOGLE_API_KEY as a Render secret environment variable.
4. Keep the persistent disk mounted at /app/data.
5. Deploy and verify the /_score/health endpoint.
6. Create a user, upload a small test document, process it, ask a question, and download an export.

The included template uses the starter plan and a 1 GB persistent disk. Increase disk size and service capacity before storing large collections or serving many users.

8. OIDC configuration

For deployed authentication, copy `.streamlit/secrets.toml.example` to `.streamlit/secrets.toml`, replace the provider values, and register both local and deployed callback URLs with the identity provider. The current code exposes a Streamlit OIDC sign-in path when authentication is configured.

```
[auth]
redirect_uri = "https://your-service.example.com/oauth2callback"
cookie_secret = "long-random-secret"
client_id = "your-client-id"
client_secret = "your-client-secret"
server_metadata_url = "https://accounts.google.com/.well-known/openid-configuration"
```

9. Persistent data layout

Path	Contents	Backup importance
APP_DATA_DIR/auth.db	SQLite users and password hashes	Critical
APP_DATA_DIR/workspaces/collections	Per-user collection data	Critical
APP_DATA_DIR/workspaces/backups	Per-user exported ZIPs	Useful
/documents.json	Persisted chunks and metadata	Critical
/index.faiss	Native vector index	Rebuildable
/chat_history.json	Saved questions, answers, and sources	Important
/file_metadata.json	Categories and tags	Important

Keep a tested backup of `auth.db`, workspace collections, chat history, and metadata. FAISS indexes are rebuildable from `documents.json` when the selected AI embedding provider is available.

10. Production security checklist

- Use OIDC or another managed identity provider for public deployment.
- Keep API keys and cookie secrets in the deployment secret store.
- Set a restrictive persistent disk and backup policy.
- Keep the current safe ZIP import behavior; never trust imported pickle files.
- Add rate limiting, roles, password recovery, and two-factor authentication before broad public access.
- Add monitoring for provider errors, storage, processing duration, and failed logins.
- Treat uploaded document instructions as untrusted content in future prompt-hardening work.

11. Verification checklist

Check	Expected result
Health check	Render or Docker responds on <code>/_score/health</code>
Authentication	Register, sign in, log out, and sign in again
Indexing	A sample PDF or TXT file becomes searchable
Citations	Answers show source numbers and source previews
Comparison	Two documents produce text diff and summary
Persistence	Restart preserves user and collection data
Backup	Exported ZIP can be imported and reindexed
Tests	pytest passes; external tests run only with provider setup

12. Troubleshooting

Gemini key error: set `GOOGLE_API_KEY` and restart the app. Ollama error: confirm the service is running and both model names are installed. OCR error: install Tesseract and verify the executable path. PDF rendering error: install Poppler utilities outside Docker. Missing search results: reprocess the collection after changing provider or OCR settings. Render data loss: verify that the persistent disk is mounted at `/app/data`.

13. Technology quick reference

Layer	Technology
Application	Python 3.11 and Streamlit
AI	Gemini google-genai and optional Ollama
Orchestration	LangChain
Retrieval	FAISS, BM25, Sentence Transformers
Parsing and OCR	pypdf, Pillow, Tesseract, pdf2image, python-docx, python-pptx
Data	SQLite, JSON, native FAISS files
Deployment	Docker, Render, GitHub Actions
Testing	pytest with optional provider integration tests